Search & Rescue (Activity Two) — Complete Extraction

Source: `Version2_SST_Makerspace_Competition_Document_2026.pdf` — *SST Makerspace 3.0, Competition Brief V2, October 2026, The Rope Runner Challenge*.

Everything below is taken from the document. Nothing has been added, interpreted, or filled in. Where the document contradicts itself, both versions are shown. Page numbers refer to the PDF's own printed page numbers where available.

---

1. Activity Two — Overview and Goal

- The activity **simulates a post-disaster search-and-rescue mission**.
- Each crawler bot must navigate along **the same rope used in Activity One**, but now performs the additional task of **identifying "victims" positioned beneath the rope** and **transmitting the necessary evidence to a local server**.
- The robot must:
  1. use its **bottom-facing camera** to visually detect victims,
  2. **capture a clear image**,
  3. **calculate the distance travelled from the start of the rope using wheel encoders**,
  4. **send both the image and its associated metadata to the competition server**.
- **The Goal:** "to locate and correctly report as many victims as possible in the least time".
- Restated goal in the Technical Build Guidelines: *reliably capture a clear image of a "victim", pair it with an accurate distance reading, transmit both to the server, and have the server (authoritatively) detect and score the submission.*

Sample victim/healthy marker images (from the Appendix): SSTMS26_Images_People.pdf — Google Drive

---

2. Victim Markers — Size, Colour, Placement Rules

- Each victim is represented by a **printed marker showing an injured person**.
- Printed on **clear white paper**.
- Size: **between 12 and 15 cm on each side**.
- Source image resolution: **700–900 px**.
- Sample images are referenced in the **Appendix ("Sample Images - Victims")** → SSTMS26_Images_People.pdf — Google Drive
- Markers will be **placed flat on the floor beneath the rope**.
- **Spaced randomly**.
- **Not rotated significantly**.
- **Always fully visible.**
- **Mixed with images of other "healthy" persons.** The document states explicitly: *"This is to test model training integrity."*

  This is the only statement in the document about the healthy/victim distinction: healthy-person images exist as distractors, and correctly not reporting them is part of the test.

---

3. Camera Requirements

From the Activity Two section:

- Camera **mounted facing directly downward**.
- Maintaining a distance of **roughly 22–28 cm from the ground**.

- Images captured in **JPEG format**.
- **Ideally QVGA (320×240)** for quick transmission.
- **VGA (640×480) may be used if a team is confident in the WiFi performance.**

From the Technical Build Guidelines (Camera placement & capture, ESP32-CAM):

- **Mounting:** rigidly mount the ESP32-CAM **directly beneath the robot, pointing straight down**. **Eliminate wobble with a short bracket + rubber damping.**
- **Height: 22–28 cm above target plane (optimum).** Verify with a test print so **the victim fills ~25–40% of the frame**.
- **Resolution/format:** default **QVGA (320×240) JPEG** for speed; **VGA (640×480) only if Wi-Fi is rock-solid**. Aim for **JPEG size <150 KB (quality ~60–75)**.
- **Lighting:** add a **small downward LED ring** to stabilize exposure.

⚠ **Document contradiction on camera height.** The Electrical & Power Design section (Sensors, Encoders & Data Integration) instead says: *"mount the camera at a fixed height (~6–10 cm above the target plane) to ensure consistent image size and focus."* Two other places say 22–28 cm. The rope itself is specified at **25 cm from the ground**, which is consistent with 22–28 cm and inconsistent with 6–10 cm. Worth raising with organizers.

Also from that section:

- The **ESP32-CAM or external camera module must be oriented downward with consistent lighting**.
- **A short ribbon cable or directly mounted ESP32-CAM board is recommended to avoid movement blur.**

---

4. Distance Measurement (Wheel Encoders)

- **For Activity 2, wheel encoders are required** to determine distance along the rope.
- As the robot moves along the rope, **its wheel encoders must keep an accurate log of the distance travelled**.
- **Competitors will compute this themselves** using the standard relationship between **encoder ticks, wheel circumference, and forward displacement**.
- **Encoders must be rigidly aligned with the drive shafts**; even small misalignments cause **pulse skipping or jitter**.
- **Use hardware interrupts** on the microcontroller to ensure accurate pulse counting at high speed.
- **Distance is computed from pulse count × wheel circumference × gear ratio.**
- **This distance must be transmitted alongside the image data for victim documentation.**

From the Technical Build Guidelines (Encoders — distance):

- Use a **dedicated friction-pulley-driven encoder**; measure and record `ticks_per_rev` and derive `distance_cm = (ticks / tpr) * circumference`.
- **Mounting:** rigid, **spring-loaded idler** to ensure consistent motor shaft contact.
- **Sync:** capture the **timestamp and the current encoder tick at the instant of exposure — these two values must travel together**.

Stated assumptions for the wheel encoder section:

- Quadrature **or** single-channel encoder.
- Encoder mounted on a **friction pulley**.
- Known `TICKS_PER_REV` and **pulley circumference**.

(The document includes a code snippet image on this page which is not machine-readable in the file; see §12.)

---

5. Image Capture, Team Tag Overlay & Metadata Specification

- **At the moment a victim appears in the frame**, the robot must capture a photograph that **clearly shows the victim** and is **free from blur or obstruction**.
- The image **must include an overlaid team tag**:
  - written in a **readable white font with a black outline**,
  - **placed neatly in one corner**,
  - **this tag must match the metadata the robot sends**.
- Once captured, the robot must **package the image in base64 format** together with:
  - **team name**,
  - **timestamp**,
  - **measured distance (in centimetres from the start of the rope)**, and **transmit everything via HTTP POST to the local server's upload endpoint**.
- **A retry system should be implemented:** if the upload fails, the robot **attempts again several times with a short delay**.
- **The server will respond with a confirmation** that the image and data have been received and accepted.

Counting criteria — an image is only counted if it:

1. **displays the victim clearly**,
2. **includes the proper team tag**,
3. **is readable as a valid JPEG**,
4. **has a plausible distance value**.

Data Structure (exactly as printed in the document)

json

```
"team_id": "Phoenix",
"victim_tag": "victim",
"distance_cm": x,
"timestamp": "2025-03-11T14:25:30Z",
"image_id": "Phoenix_142530_234cm.jpg",
"image_base64": "<base64>"
```

Note the `image_id` naming pattern implied by the example: `TeamName_HHMMSS_<distance>cm.jpg`.

Mandatory metadata (Technical Build Guidelines version — with every image)

- `team_id` (string)
- `distance_cm` (numeric, from encoders)
- `capture_timestamp` (ISO8601)
- `image_id` (unique per image)
- `image` (binary JPEG **or** base64)

⚠ **Field-name contradiction between the two sections:** Activity Two lists `timestamp` / `image_base64` / `victim_tag`; the Technical Build Guidelines list `capture_timestamp` / `image`. The Activity Two JSON block is the one presented as *the* Data Structure. Confirm with organizers / mock server.

Image pre-processing

- **Pre-send checks:** basic **brightness/blur check**; **crop to ROI if possible to reduce size**.
- **Atomicity: send image + metadata in one POST; never separate them and expect the server to match later.** ("Key idea: Image and metadata must be sent together in one request.")

---

6. Communication Protocol & Reliability

- **Transport:** HTTP POST to `/upload_detection` (**multipart/form-data preferred**) on the **local server**.
- **Include header: `X-API-Key: <teamkey>`.**
- **Official API Key will be sent out after mock qualification.**
- **Retry policy: exponential backoff up to 5 attempts** (e.g., **0.5 s, 1 s, 2 s, 4 s, 8 s**). On persistent failure, **queue to local storage (SPIFFS/SD) for bulk upload after the run**.
- **Limits:** keep images **<150 KB** and **avoid flooding — rate-limit the client to 1–2 uploads/sec**.
- **Time sync:** the **ESP32 should NTP-sync at boot**; **server time is authoritative for disputes**. Always include the **device `capture_timestamp`**, and the **server will add `receive_timestamp`**.
- From the electrical section: for SAR image transmission, **the ESP32 or Raspberry Pi should form an HTTP/MQTT client** sending **bundled JSON + image payloads** to the local server.
- **Server details will be communicated to qualified teams after the mock activity.**

⚠ **Contradiction on encoding:** Activity Two says base64 inside a JSON body; the Technical Build Guidelines prefer `multipart/form-data` with binary JPEG. Both appear. Resolve against the actual mock server.

---

7. Model & Training Essentials (as specified by the document)

- **Task:** single class **"victim"** — *keep classes minimal*.
- **Data collection:** use the **same camera and mount as competition**. Collect **300–500 labeled images minimum (per class)**. Include **20–30% negative frames (no victim)**.
- **Annotation: tight, consistent bounding boxes**; use **LabelImg / CVAT / Roboflow**.
- **Split: 70% train / 20% val / 10% test**; **avoid session leakage** (don't split near-duplicate frames across sets).
- **Augmentation: moderate brightness, slight rotation (±10°), motion blur, scale**. **Avoid unrealistic transforms.**
- **Model choices:** lightweight detectors — **YOLOv5n / YOLOv8n / MobileNet-SSD**. Input size **320×320 or 640×640 depending on server capacity**.
- **Training:** monitor **precision, recall, mAP**; **favor recall** (*missed victims worse than false positives*).
- **Thresholding:** confidence **~0.4–0.6**; **require minimum bbox area to reject noise**.
- **Validation: test under motion** — run full robot captures, upload to server, **verify detection rates in real conditions**.

  Note: "input size 320×320 or 640×640 **depending on server capacity**" and the goal statement "have the **server (authoritatively) detect and score** the submission" both imply the *organizers'* server runs the scoring detection. Whether teams are expected to also run detection on-board is never stated outright — but the mock video checklist (§10) requires showing "the robot capturing an image **and identifying it**", which implies on-board identification.

---

8. Scoring — Search & Rescue

Category — Search & Rescue: 40% of the competition (largest single category).

Sub-category	Max Points
Number of Victims Detected (aggregate)	80
Accuracy & Tagging Quality (aggregate)	40
Reliability of Transmission (aggregate)	20
Speed Bonus (mission completion time)	20
Search & Rescue Subtotal	**160**

For context, the full scoring system:

Category	Weight	Subtotal
Design	25%	100
Swiftplay	35%	140
Search & Rescue	40%	160
TOTAL		**400**

(Design breakdown: Chassis Design 20, Drive Mechanism 20, Aesthetics & Presentation 20, Innovation/Extra Features 20, Obstacle Navigation 10, Electronics & Wiring 10. Swiftplay breakdown: Traversal Time 100, Obstacle Handling & Stability 25, Completion Status 15.)

SAR-relevant bonus

- **First-Attempt (SAR Bonus): +5 points per victim** (within SAR scoring) **if the image was transmitted and accepted on the first upload attempt**. **Capped at +15 points across all victims.**
- **Maximum Bonus Cap:** a team may receive **up to +20 bonus points total** across all bonuses; beyond this, additional bonuses are not counted.

Tie-breakers (SAR is first)

1. **Higher SAR subtotal** (team with better search & rescue performance wins).
2. Higher Swiftplay Traversal Time ranking (faster traversal).
3. Higher Innovation/Extra Features score.
4. Fewest falls across Swiftplay.

---

9. Activity Two — SAR Penalties

Violation	Penalty
Blurry / unusable image	**Image rejected (0 points)**
Victim partially visible	**−5 points per image**
Incorrect victim label	**−10 points per image**
Missing label	**−10 points per image**
Missing team name/ID	**Image rejected**
Manual image upload / screenshot	**Submission invalidated**
Incorrect data format	**−10 points**
Any intentional cheating	**Immediate disqualification**

General penalties that also apply during SAR

Violation	Penalty
Unsafe design (exposed wiring, overheating battery, loose parts)	−30 points
Minor rule violation (non-critical spec breach)	−15 points
Major rule violation (size, rope damage, prohibited parts)	−50 points
External assistance during run	−40 points
Unsportsmanlike conduct (minor)	−20 points
Failure to fix safety issue after warning	Disqualification
Severe misconduct or disruption	Disqualification

---

10. "Other Info" — Success Conditions (verbatim content of that subsection)

To succeed in this activity, each team must ensure that their robot can:

- **reliably capture images**,
- **correctly overlay the team tag**,
- **measure distance accurately**,
- **communicate with the server without corruption**.

A **valid entry** requires that:

- the **victim fills enough of the frame to be recognisable**,
- the **team information matches across both the image and metadata**,
- the robot **maintains logical forward progression in its recorded distances**.

Additional notes:

- **Lighting may vary slightly across the venue**, so teams should **avoid relying on auto-exposure alone** and **consider adding soft downward LEDs for stability**.
- **Image compression should be balanced:** *too high causes blur, too low results in long upload times.*
- Teams should **thoroughly test their JSON formatting and encoder calibration before competition day**.
- **Server details will be communicated to qualified teams after the mock activity.**

---

11. SAR Requirements in Deliverables & Qualification

Mock Selection video (required content — SAR items)

The video must show, among other things:

- **a short overview of the software logic**,
- **the robot capturing an image and identifying it**,
- **the robot sending the image over a local network (to a locally hosted server)**.

Other video requirements: robot at rest; robot being measured; robot being attached to the rope; robot powered on; robot moving forward on the rope; gripping mechanism in operation; at least a short traversal segment proving the motion is real and controlled. Resolution **1920×1080**, clear, stable, high-resolution. **Mock Event / Video Submission Deadline: 17 August 2026.**

Technical Specification Sheet — Software Section

Must explain:

- the control logic used for movement,
- whether the bot is autonomous or semi-autonomous,
- **how encoder data is read**,
- how the bot decides when to move or stabilize,
- **any sensor use**,
- **how the system handles timing, retries, or image/data transmission if your bot uses SAR-related features**.

Review 4 (July 8)

Content requirement includes: **"Camera/image capture setup if SAR is involved"**, alongside successful rope traversal tests, stability/grip results, **distance or timing test results**, software/debugging status, final list of unresolved issues.

Suggested BOM — SAR-relevant items

- **Sensors:** Camera Module (e.g., **ESP32-CAM, Raspberry Pi Camera**) — *for FPV or image processing of images*; **Wheel Encoder for location sensing**.
- **Connectivity: Wi-Fi Module (ESP32 built-in)** — for web-based control or data streaming; Bluetooth Module (HC-05/06) for smartphone control.
- **Control System:** Microcontroller — Arduino Uno, ESP32, or Raspberry Pi; Motor Driver L298N or similar H-bridge; Power supply rechargeable LiPo (e.g. 7.4 V, 1000 mAh).
- **Software:** Arduino IDE or VS Code with PlatformIO; Python for Raspberry Pi applications.

---

12. Architecture, Task Scheduling & Compute (SAR-relevant)

- **The onboard architecture must handle task scheduling: motor control loop, encoder counting, and camera capture must not block each other.**
- **On ESP32, dual-core allocation or timed tasks (e.g., FreeRTOS) are ideal. On Arduino, offload camera to a separate MCU.**
- **Recommended Control Architectures:** *"A dual-processor setup is ideal: an ESP32 or Arduino handling motion + encoders, and an ESP32-CAM handling imaging and network uploads. For advanced teams, a Raspberry Pi Zero 2 W can unify everything but must be thermally managed and given a stable 5 V supply. The final architecture must ensure deterministic timing for motor control and data reporting."*
- From the System Architecture Overview: in the SAR phase, *the software shifts to an intelligent surveillance role, **fusing visual data from the camera with location coordinates from the encoder** to identify, tag, and transmit images of "victims" to a local server using Wi-Fi.* Integration is achieved when the software processes sensory inputs in real time to adjust mechanical torque and speed, **ensuring the bot remains stable and responsive while executing complex data transmission tasks without pausing its journey**.
- The microcontroller *"integrates sensory inputs from a bottom-facing camera and a wheel encoder to create a digital understanding of the environment below the rope."*
- **Autonomy & Control rule (applies to SAR):** full autonomy is **not** required; **remote control (Bluetooth, RF, Wi-Fi) is allowed during operation**; **all logic must be on-board (microcontroller, sensors, encoders)**; permitted actions include **pre-loaded code** and **autonomous adjustments guided by sensors**.

  ⚠ Note the phrase **"All logic must be on-board"** under Autonomy & Control. This is the clause that bears directly on whether classification may be offloaded to a team-side laptop. It is not explicitly reconciled anywhere with the "server capacity" language in the training section. **This needs organizer clarification.**

Code snippets not extractable

Two pages contain code as **rendered images rather than selectable text**, so their contents could not be transcribed from the file:

- The **wheel encoder (distance measurement)** snippet.
- The **atomic image + metadata upload (HTTP POST)** snippet.

Only the surrounding prose (captured above) is machine-readable. These should be read directly from the PDF.

---

13. Course & Environment Facts Relevant to SAR

- **Rope length: 8 metres**, suspended between two poles — the same rope as Activity One.
- **Rope distance from ground: 25 cm.**
- Rope type: **synthetic braided (nylon or polyester)**, diameter **12–16 mm**.
- Rope is **firmly tensioned between two strong poles**, at a height **ensuring clear visibility**, and **slightly inclined or horizontal depending on event setup**.
- **Vibrations are introduced at specific moments** to simulate real-world instability; **all teams experience the same vibration pattern**.
- **Lighting: consistent, controlled indoor lighting.**
- **Checkpoints** placed at fixed distances, used as restart locations if a bot falls.
- **Fairness Rule:** all teams run under **identical course conditions (rope tension, obstacle placement, lighting)**; any deviation is recorded and compensated uniformly.
- **Revisions could be made to the rope characteristics but will be communicated.**

Run rules affecting a SAR attempt

- Each team receives **one official run per activity** and **one practice run before the event**.
- **Retry** permitted only for battery failure, motor breakdown, or unexpected malfunction — **at judges' discretion**, with a **5-minute limit** to reset and restart. (FAQ clarifies: reserved for genuine mechanical/electrical failures, **not code errors**.)
- **Battery-operated only**; no external power cables, tethered supplies, or wired connections during competition.
- The robot must **fit within 25 cm × 25 cm × 25 cm before deployment** — the camera mount, LED ring, and any bracket count toward this.
- **Physically touching the robot mid-run is disallowed.**

---

14. Disputes Involving SAR Data

- Appeals must be lodged **immediately after posting of official results** at the Judges' table.
- Appeals must state the reason and **provide evidence if available (camera log, local video clip)**.
- **Judges convene within 15 minutes to review recorded video and server logs.**
- A replay may be scheduled if necessary; **limited to one per appeal**.
- Written, timestamped response within **30 minutes** of filing.
- **Judges' decisions are final**; reconsideration only for procedural errors, not subjective judgment calls.
- **Server time is authoritative for disputes** (see §6).

---

15. Open Questions Raised by the Document Itself

These are genuine ambiguities in the source, listed so they can be taken to the organizers:

1. **Camera height:** 22–28 cm (Activity Two + Technical Build) vs ~6–10 cm (Electrical section). Rope is at 25 cm from the ground.
2. **Payload format:** base64 in a JSON body (Activity Two) vs multipart/form-data with binary JPEG (Technical Build).
3. **Field names:** `timestamp` vs `capture_timestamp`; `image_base64` vs `image`. Is `victim_tag` required in the multipart version?
4. **Where detection runs:** "server (authoritatively) detects and scores" and "input size depending on **server capacity**" vs "**All logic must be on-board**" and the mock video requirement to show "the robot capturing an image **and identifying it**". Is a team-side offboard inference machine permitted?
5. **Endpoint:** Activity Two says "the local server's upload endpoint"; Technical Build says `/upload_detection`. API key and server details come **after mock qualification**.
6. **Healthy-person markers:** no penalty line explicitly covers *uploading a healthy person as a victim* — the closest is "Incorrect victim label −10 points per image". Confirm whether a false positive is scored as −10 or as something harsher.
7. **Qualification count:** the Selection Process says **top 15 teams**; the FAQ says **top 10 teams**.

---

16. Appendix Items Referenced (SAR-relevant)

- **A – Documents:** Judge Scoring Sheet; Mock Race – Evaluation Rubric; **Sample Images – Victims →** SSTMS26_Images_People.pdf — Google Drive
- **B – Resources:** Judge Scoring Rubric; Building a Crawler Bot; Inspiration Video for Building Bots.

(The remaining appendix items are hyperlinked in the original PDF. The copy of the file in this project stores each page as a flattened image plus a plain-text layer, so those URLs were not preserved and could not be extracted — open the original PDF to retrieve them.)

> The Sample Images are the single most important linked asset for SAR: they define what the victim markers actually look like, and they are the reference for the "injured person vs. healthy person" distinction that the whole classification task rests on. Get these before collecting any training data.

---

17. Key Dates

Event	Date
Registration form closes	15 April 2026
Review 1 – concept validation	27 April 2026
Review 2 – video submission + pledge form	15 May 2026
Review 3 – structural build & integration	17 June 2026
Review 4 – functional subsystem testing (**camera/image capture setup if SAR is involved**)	8 July 2026
Mock Event / Video Submission Deadline	17 August 2026
Main Event (D-Day)	5 – 9 October 2026 (tentative)

Contact: **sstmakerspace@pau.edu.ng** · Instagram: **@sstmakerspace** Venue: **SST Foyer, Pan-Atlantic University, Ibeju-Lekki, Lagos.**